

Guarantee B: Same Substrate State Plus Same Orchestration Rules Yields Rule-Equivalent Cell Behavior — A Standalone Treatment of Cell-Behavior Determinism in the CKS Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate Guarantee B of the CKS determinism contract — the cell-behavior-determinism commitment that same substrate state plus same orchestration rules produces rule-equivalent cell behavior — as a standalone architectural commitment that can be defended, implemented, and tested independently of the contract's other components.

Abstract

The CKS determinism contract (parent note A1.10) commits to five guarantees over the substrate. The integrating-frame note A2.55 enumerated them; A2.56 formalized the substrate/model boundary the contract operates within; A2.57 formalized Guarantee A (read-determinism). This note formalizes Guarantee B — same substrate state plus same orchestration rules produces rule-equivalent cell behavior — as having independent architectural content with particular weight on the *rule-equivalent* framing that distinguishes the guarantee from naive deterministic-execution requirements current LLM technology cannot satisfy. The note states the commitment as four operational components (equivalence at substrate-write level, at conflict-preservation level, at provenance level, and rule-bounded variation at the non-substrate-output level), explains the rule-equivalent framing across the labor-allocation framework, distinguishes Guarantee B from four adjacent computation-determinism patterns commonly conflated with it, identifies ten failure modes, and gives an operational test for whether a system's cell-behavior architecture satisfies the commitment.

1. Why Guarantee B needs to be formalized as standalone

The integrating-frame note A2.55 enumerated the five guarantees of the determinism contract but did not specify each in standalone form. A2.56 formalized the substrate/model boundary the contract operates within. A2.57 formalized Guarantee A — the read-determinism that makes substrate-mediated reads reproducible. This note formalizes Guarantee B — cell-behavior-determinism — as having independent architectural content. Guarantee A and Guarantee B compose at the cell-execution boundary (A2.10): Guarantee A makes the cell's read of the substrate deterministic; Guarantee B makes the cell's behavior, given that read, rule-equivalent across executions.

The motivating cases are deployments in which cell behavior under specified rules must be operationally predictable across executions. Regression tests verify cell behavior at a fixed substrate state and rule set (the

operationalization is the subject of A2.63). Reasoning about a system's behavior across deployments using the same orchestration rules depends on the same property. Explaining cell behavior to humans exercising governance under A1.01 requires that the behavior be reconstructable from the substrate state and the rules, not from per-execution incidental detail. Each scenario depends on Guarantee B in its rule-equivalent form.

Guarantee B forecloses two opposed architectural mistakes: over-claiming cell-behavior determinism by requiring bit-identical cell outputs (operationally infeasible for non-deterministic LLMs and incompatible with the AI-as-substrate-mediator commitment in A1.04); and under-claiming it by allowing arbitrary cell-behavior variation (which severs rule-governance from architectural consequence). The rule-equivalent framing is what threads between them.

Compositionally, Guarantee B sits between Property B (A2.20) — LLM writes occur under orchestration rules — and A2.62 category (a), which names LLM token output as the permitted non-determinism. Guarantee B specifies the determinism property Property B's rule-governance produces, and is what A2.62 category (a) is accommodated within. Property B makes the LLM's write rule-structured; Guarantee B makes the cell's behavior rule-equivalent across executions.

2. The Guarantee B commitment, defined precisely

Guarantee B states: for any substrate state S and orchestration rule R, two cells executing on S under R produce rule-equivalent behavior. The commitment decomposes into four operational components.

(a) Equivalence at substrate-write level. The cells produce substrate writes that are rule-conformant. The writes match the structure R specifies — which fields are written, which entities are referenced, which relationships are established. The writes satisfy R's constraints — validity, format, and content rules. The writes occur at the substrate scopes R authorizes. Specific content within rule-permitted variation may differ across executions; structural conformance to R does not.

(b) Equivalence at conflict-preservation level. The cells produce conflict-preservation behavior that is rule-determined. When the cell encounters contradicting content, it preserves the contradiction at the substrate level (per A2.13) using the relationship metadata R specifies. Resolution decisions, where R's logic permits resolution under specified conditions (per A2.14), follow R's resolution logic deterministically. Conflict provenance (per A2.16) conforms to R's provenance schema. Different executions produce the same preservation pattern, the same resolution-or-non-resolution decision, and the same provenance shape.

(c) Equivalence at provenance level. The cells produce provenance (per A2.40) that is rule-conformant. Writer attribution identifies the cell and its rule; timestamp records the moment of write; antecedent reference identifies what substrate content the cell read; rule reference identifies R; rationale, where R requires it, is recorded with the structure R specifies; relationship metadata, where applicable, is recorded per R's relationship schema. The provenance is rule-deterministic at the structural level. Specific content of fields R permits to vary — most notably LLM-generated rationale text — is allowed to vary within R's bounds.

(d) Rule-bounded variation at the non-substrate-output level. The cells may produce non-substrate-side outputs — LLM token streams that do not write to substrate, intermediate computational artifacts, log entries — that vary across executions. The architectural commitment is that this variation is bounded to rule-allowed limits and does not propagate to the substrate side. Components (a)–(c) are not weakened by component (d): non-substrate variation is admissible specifically because, by component (d)'s scope, it does

not touch what the substrate carries.

The four components together specify what cell-behavior-determinism is in CKS. A system that satisfies all four has the property; a system that fails any one does not, regardless of how reliably it appears to behave in any given session.

3. The “rule-equivalent” framing

The choice of *equivalent* rather than *identical* in Guarantee B is load-bearing, and the distinction is the architectural commitment this note's standalone treatment is most concerned with.

Equivalent means rule-conformant at the substrate-side, not bit-identical at the execution level. It requires rule-conformance at the structural level: two cell executions on the same substrate state under the same rule produce writes that match the rule's structure, write to the same substrate scopes, satisfy the same constraints, and record provenance with the same schema. It requires rule-determined behavior at substrate-side decisions: conflict preservation, resolution under rule-specified conditions, and provenance recording follow the rule's logic deterministically — the cell's decisions about what to preserve, how to resolve, and what provenance to record are rule-determined, not LLM-determined.

It does *not* require bit-identical substrate writes. Two executions may produce writes whose specific text differs — different LLM-generated rationale phrasing, different ordering where order is not rule-fixed — provided every write conforms to the rule's specification. It does not require equivalence at the LLM-output layer; LLM token streams may differ across executions, and what the rule binds is the substrate-side output, not the model-output layer (the boundary A2.56 formalizes). It does not require equivalence at the non-substrate-output layer; intermediate artifacts and logs may vary within rule-allowed bounds.

The framing accommodates the full labor-allocation framework of A1.12, not only the LLM-cell case. Stable cells (Mode 3) executing rule-determined logic without LLM involvement produce equivalence that approaches bit-identity — the cell's output is determined by the rule's logic and the substrate state, with no non-deterministic component. Cells under direct human labor (Mode 1) produce equivalence at the rule-conformance level mediated by human compliance with the rule. Cells executing LLM-under-rule labor (Mode 2) produce equivalence at the rule-conformance level at the substrate-side, with LLM non-determinism absorbed into component (d). All three modes satisfy Guarantee B; the architectural property is uniform across the labor framework, even though the *degree* of bit-level equivalence varies by mode.

The framing is operationally compatible with current LLM technology because it does not require LLM determinism, and architecturally rigorous because it requires rule-conformance at every substrate-side boundary downstream commitments depend on — provenance for retraceability per A1.07, preservation for conflict-handling per A1.03, rule-conformance for governance per A1.01.

4. What the guarantee does not claim, and what it is not

Six things Guarantee B does not claim. It does not claim LLM token outputs are deterministic; LLM non-determinism is allowed under A2.62 category (a) and is accommodated through the rule-equivalent framing. It does not claim cells execute in zero time; the commitment is to rule-equivalence, not execution-performance. It does not require all cells to use LLMs; stable cells (A1.12 Mode 3) are admissible and produce equivalence closer to bit-identity. It does not specify enforcement patterns for rule-conformance; schema validation, rule-engine-driven structuring, post-hoc validation with retry, and other mechanisms are deployment choices. It does not foreclose cells producing diverse outputs within rules; only outputs violating the rule's specification fail the guarantee. It does not specify cell statelessness across executions; statelessness is a separate commitment under A2.09, with which Guarantee B is compatible but does not subsume.

Four adjacent computation-determinism patterns Guarantee B is not.

Not deterministic execution semantics. Deterministic execution semantics requires program execution to produce bit-identical outputs across identical-input executions. Guarantee B specifies rule-equivalence at substrate-side, not bit-identity at execution level. Stable cells may approach deterministic-execution semantics; LLM cells may satisfy Guarantee B without satisfying it.

Not bit-identical computation. Bit-identical computation requires same-bit outputs across executions. Guarantee B does not require this — LLM token outputs may differ bit-wise across executions while substrate-side outputs satisfy rule-equivalence.

Not algorithmic reproducibility. Algorithmic reproducibility requires an algorithm to produce the same output for the same input. Guarantee B is at a different layer: rule-equivalent cell behavior, not algorithm-level reproducibility. The cell's algorithm may be reproducible (stable cells) or non-reproducible (LLM-driven cells); rule-equivalence is independent of either.

Not replay-equivalent behavior. Replay-equivalent behavior requires a system, given the same inputs in sequence, to produce the same outputs in sequence. Guarantee B is broader: rule-equivalence regardless of execution sequence, timing, or replay context. A non-replay-equivalent system may still satisfy Guarantee B if rule-conformance holds.

5. Failure modes that violate the guarantee

Ten anti-patterns recur in implementations that approximate Guarantee B without satisfying it.

(a) LLM-driven substrate writes without rule structuring. The implementation permits LLM-driven cells to write to substrate without rule-governance per Property B (A2.20). Cell writes vary across executions in ways not rule-bounded; substrate-side outputs are not rule-equivalent.

(b) Rule-bypass execution paths. Alternative execution paths — direct-write APIs, admin-override mechanisms not modeled as rule-governed cells — bypass orchestration rules. Cell behavior under these paths is not rule-governed and therefore not rule-equivalent.

(c) Rule non-determinism. The orchestration rules themselves incorporate non-deterministic elements — randomization, time-dependent logic, external-source dependencies that vary across executions. Same state plus same rule produces different rule semantics across executions; rule-equivalence cannot hold because

“same rule” does not produce same behavior.

(d) Rule mutation during cell execution. Orchestration rules are modified mid-execution (hot-reload, runtime rule updates). Cell behavior reflects different rules across executions; “same rule” is not preserved across the executions being compared. Rule modifications properly belong to design time per A2.04, not to runtime.

(e) LLM-output-as-rule-substitute. The implementation treats LLM outputs as authoritative for behavior structure rather than the rule — LLM-generated decisions about what to write, how to handle conflicts, what provenance to record. The LLM's non-determinism propagates to substrate-side outputs; rule-equivalence fails because the LLM, not the rule, is determining the structure the substrate carries.

(f) Conflict-resolution non-determinism beyond rule specification. Cell conflict-resolution decisions vary across executions in ways not bounded by the rule (e.g., LLM-driven resolution choices that exceed R's resolution logic). Component (b) fails.

(g) Provenance non-determinism beyond rule specification. Cell provenance recording varies in ways not bounded by R — random ordering of provenance fields, non-deterministic timestamp generation, varying writer attribution. Component (c) fails.

(h) Non-substrate-output propagation to substrate. Non-substrate outputs propagate to substrate without rule-structuring. Non-substrate non-determinism becomes substrate non-determinism; component (d)'s containment fails.

(i) Rule-engine non-determinism. The rule engine itself produces non-deterministic interpretations of rules — probabilistic rule matching, fuzzy rule application. Same rule produces different rule semantics across executions; rule-equivalence fails at the rule-interpretation level.

(j) Cell-internal-state variation. The cell carries internal state across executions that varies — learned parameters, accumulated context, behavioral drift. The cell's behavior is not rule-equivalent because cell-internal state introduces non-rule-governed variation. (This anti-pattern composes with A2.09's statelessness commitment and is named here as the determinism failure that follows from violating it.)

In each of these patterns the system may behave usefully in any given session. What it loses is rule-equivalence, and through it, the architectural predictability downstream commitments depend on.

6. Operational test

A system satisfies Guarantee B if and only if all of the following are true at all times during the substrate's existence:

1. Rule-conformant substrate writes. For any substrate state *S* and orchestration rule *R*, two cells executing on *S* under *R* produce substrate writes whose structure, scope, and constraints match *R*'s specification.

2. Rule-determined conflict behavior. Cells produce conflict-preservation, resolution-or-non-resolution, and conflict-provenance behavior determined by *R*, not by the LLM and not by per-execution incidental factors.

3. Rule-conformant provenance. Cells produce provenance whose structural fields — writer, timestamp, antecedent, rule reference, rationale where required, relationship metadata where applicable — conform to R's schema.

4. Bounded non-substrate variation. Variation in non-substrate-side cell outputs is bounded to rule-allowed limits and does not propagate to substrate-side outputs.

5. Rule stability across executions. R is stable across the executions being compared; rule modifications occur at design time per A2.04, not during the executions in question.

6. Verifiability through regression testing. The above is checkable by fixing substrate state and rules and observing rule-equivalent cell behavior across executions; the regression-testing operationalization is the subject of A2.63.

A system that fails any of (1)–(6) does not satisfy Guarantee B in the architectural sense, even if its cell-execution functions operationally.

7. Why naming Guarantee B as standalone matters

Implementations under pressure to deliver flexible LLM-driven AI behavior consistently drift toward two opposed failure patterns: over-claiming determinism by requiring LLM determinism (which current LLM technology cannot supply), or under-claiming it by allowing arbitrary cell-behavior variation (which makes orchestration rules advisory and decouples cell behavior from rule-governance). Both drifts share a root cause: the rule-equivalent framing is operationally subtle, requiring the implementer to accommodate LLM non-determinism while still binding substrate-side outputs to rule-conformance.

Naming Guarantee B as a standalone architectural commitment — with the four operational components in §2, the rule-equivalent framing across the labor-allocation framework in §3, the boundaries and adjacent-pattern distinctions in §4, the failure modes in §5, and the operational test in §6 — gives implementers, auditors, and downstream architects a single specification of what cell-behavior-determinism in CKS is.

The downstream commitments that depend on Guarantee B make the standalone formalization load-bearing. Without it, the integrating contract A1.10 cannot operate at the cell-execution boundary; the AI-as-substrate-mediator commitment A1.04 and Property B (A2.20) lose their determinism property, with rule-governance becoming an unenforced annotation on the LLM's writes; the path-retraceability commitment A1.07 cannot reconstruct cell behavior from the rule reference; regression testing per A2.63 has no determinism property to verify; the labor-allocation framework A1.12 cannot guarantee architectural predictability for Mode 2 (LLM-under-rule) and Mode 3 (stable-cell) labor; and rule authoring as governance per A2.04 produces no architecturally bound runtime effect.

The subsequent decomposition notes — A2.59 (Guarantee C), A2.60 (Guarantee D), A2.61 (Guarantee E), A2.62 (allowed non-determinism), A2.63 (regression testing) — specialize the remaining components of the contract. Together with A2.55–A2.58, they give the full operational decomposition of A1.10. Subsequent work that adopts the CKS pattern, extends it, or argues against it should engage with Guarantee B's rule-equivalent framing as specified here. Subsequent work that uses *cell-behavior-determinism* in a different sense — bit-identity, replay-equivalence, algorithmic reproducibility — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Guarantee B: Same Substrate State Plus Same Orchestration Rules Yields Rule-Equivalent Cell Behavior — A Standalone Treatment of Cell-Behavior Determinism in the CKS Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.